Smart City Traffic Analytics: Machine Learning & Deep Learning Project Report

1. Project Overview & Architecture

The goal of this project is to build an end-to-end traffic forecasting and pattern recognition system using over 48000 hourly records of metropolitan traffic and weather observations.

To move from raw sensor logs to clear, explainable predictions, the work followed three main phases:

- **Supervised Machine Learning**: Built baseline tabular models to predict exact vehicle counts and classify congestion levels.
- **Unsupervised Learning & Pattern Discovery**: Grouped traffic behaviour into distinct operational patters and extracted practical "if-then" rules to guide dispatching decisions.
- **Sequential Deep Learning & Explainability**: Used recurrent neural networks to capture 24 hour time patterns followed by Shapley values (SHAP) to explain decisions made by model.

2. Technical Stack & Core Libraries

The project used the following core Python libraries across all phases

- **Data Engineering & Preparation**:
  - pandas & numpy: Cleaned data, sorted records chronologically, split data into train/validation/test sets, and created cyclical sine/cosine time features.
  - scikit-learn (StandardScaler): Scaled features and targets to keep numerical training stable and speed up convergence.
- **Classical Machine Learning (Supervised & Unsupervised)**:
  - scikit-learn: Trained linear and logistic regressions, decision trees and random forests; ran l-means clustering with silhouette scores, and calculated evaluation metrics (MAE, RMSE, R^2, Accuracy and Confusion Matrices).
  - mlxtend: Ran the Apriori algorithm to discover frequent traffic patterns and generate association rules
- **Deep Learning & Neural Networks**:
  - PyTorch (torch, torch.nn, torch.utils.data): Implemented stacked LSTM networks, created sliding-window dataset generators (Dataset, DataLoader), configured loss functions and optimisers.
- **Model Explainability & Transparency**:
  - shap: Ran GradientExplainer to attribute LSTM predictions back to specific input features across time.
- **Visualizations & Reporting**:
  - matplotlib & seaborn: Plotted training loss curves, cluster boundaries, feature importance charts, and individual SHAP attribution plots.

3. Methodology & Implementation by Task

Task 1: Supervised Learning (Tabular Modelling)

High-Level Approach:

- Two models were built. One predicts how many vehicles use the road each hour. The other estimates accident risk. There was no real accident data, so a simple risk label was created from the available data: an hour counts as "high risk" when traffic is heavy and the weather is bad (snow, thunderstorm, squall, fog, mist, smoke or haze). About 11% of hours are high risk. This label is only to show how the classification works. It does not predict real accidents.
- The data was split randomly: 80% to train the models and 20% to test them.

Key Findings:

- Linear baselines underperformed because traffic patterns follow distinct cyclical daily curves that simple linear models fail to capture.
- Random Forest was the best model. It cut the average error from about 839 vehicles per hour (simple linear model) to about 268. For the risk label, it got 98% of hours right. These risk scores look high partly because the label is built from the same weather and traffic data that the model uses.

Task 2: Unsupervised Learning & Pattern Extraction

High-Level Approach:

- Discovered underlying traffic patterns without ground-truth labels using two complementary methods: K-Means Clustering and Apriori Association Rule Mining.

Key Findings:

- The Elbow Method and Silhouette analysis identified four practical traffic regimes:
    - Regime 0 (Night/Off-Peak): Light traffic and cooler temperatures, midnight to 5:00 AM.
    - Regime 1 (Steady Daytime): Moderate flow representative of typical midday and weekend traffic.
    - Regime 2 (Commuter Peak): high vehicle volumes during morning and evening rush hours.
    - Regime 3 (Adverse Weather Flow): Highway capacity reduced by significant rain, snow or fog.
- Association rule mining via “mlxtend” produced concrete operational rules, demonstrating that standard weekday peak hours link to high congestion tiers with over 80% confidence, while adverse weather during peak periods reliably signal bottlenecks.

Task 3: Deep Learning (PyTorch LSTM) & SHAP Explainability

High-Level Approach:

- Traffic patterns build up gradually over time, which isolated hourly models often miss. To capture this continuous momentum, a PyTorch Long Short-Term Memory (LSTM) network was implemented using a rolling 24-hour lookback window (T=24, F=21)
- Input features included cyclical time encodings (hour_sin, hour_cos, dow_sin, dow_cos), calendar indicators (is_weekend, is_holiday), and environmental conditions (temp, clouds_all, one-hot encoded weather categories).
- Data was fed through a custom sequence dataset paired with PyTorch data loaders. Training used the Adam optimiser ($\eta = 10^{-3}$), with MSE loss, learning rate scheduling via ReduceLROnPlateau , gradient clipping, and early stopping prevent overfitting.

Key Findings:

- Unseen Test Set Performance:
  - R^2 Score: 0.9236 (explaining 92.36% of variance in traffic flow).
  - Mean Absolute Error (MAE): 355.84 vehicles/hour on an interstate corridor carrying up to 7,000+ vehicles/hr.
  - Root Mean Squared Error (RMSE): 548.46 vehicles/hour.
- Model Explainability with SHAP:
  - Evaluated feature contributions using shap.GradientExplainer across 2,400 input points (24 hours sequences across 100 sample windows).
  - Global Insights: Time-of-day features had the strongest influence on volume forecasts, followed by weekend status and temperature. Severe conditions like thunderstorms, snow and dense fog consistently pulled predicted volumes down.
  - Local Instance Tracking: Individual forecast explanations confirmed that peak-hour timing drove predictions upward from an average of 2,934 to 5,057 vehicles/hr, while concurrent thunderstorms noticeable dampened the expected peak.

Task 4: Advanced AI Technique — MLOps Experiment Tracking with MLflow

1. **Technique Selection Rationale**

- Tracking Different Model Types: The project tests completely different modelling approaches: basic linear regression, tree ensembles (Random Forest) and Deep learning sequence models (PyTorch 2-layer LSTM). Manually comparing such different architecture leads to missing hyperparameter records, scattered evaluation numbers, and code that is difficult to reproduce. MLFlow solves this by keeping all runs, settings, and outputs organised in one place.
- Instead of relying on standalone training scripts, MLFlow introduces a clean production workflow. It automatically logs model parameters, git commits hashes, saved binaries, and scoring metrics in real time during training.

- Standard Model packaging and registry: Beyond logging scores, MLFlow saves models from different libraries (mlflow.sklearn and mlflow.pytorch) into a standard package format. For each of the save run, the environment dependencies and expected input format are bundled, making it easy to drop the model straight into an API

2. Implementation Methodology

- Database Storage: The tracking system uses a local SQLite database (sqlite:///part3_machine_learning/mlflow.db) rather than plain file folders. This ensures reliable data storage and fully supported for MLFlow capabilities
- Unified Workspace: All modelling runs are grouped under one central experiment name called SmartCity_Traffic_Volume_Forecasting.
- Automated Run Scripts (track_experiments_mlflow.py)
    -
- **Automated Run Script (track_experiments_mlflow.py)**:
    - Run 1 — Baseline Linear Regression: Fitted on 21 weather and time features. Logged model type (LinearModel), feature count (21), evaluation scores, and the Scikit-learn artifact:
        - MAE: 844.86 vehicles/hr
        - RMSE: 1,081.68
        - R^2: 0.7027
    - Run 2 — Random Forest Regressor: Fitted using 100 trees and a max depth of 18. Logged tree hyperparameters (n_estimators: 100, max_depth: 18), performance metrics, and the Scikit-learn model artifact:
        - MAE: 296.62 vehicles/hr
        - RMSE: 511.91
        - R^2: 0.9334
    - Run 3 — PyTorch Stacked LSTM: Evaluated on sequential 24-hour window (seq_len=24, hidden_dim=64, num_layers=2, dropout=0.2). Logged network settings, a sample input tensor to define the expected data shape, PyTorch model artifacts and evaluation metrics:
        - MAE: 355.05 vehicles/hr
        - RMSE: 551.61
        - R^2: 0.9227
- Dashboard verification: All runs, metrics, and saved artifacts were verified through the local MLFlow dashboard on port 5001 (mlflow ui --backend-store-uri sqlite:///part3_machine_learning/mlflow.db --port 5001).

3. Business & Operational Value

- The centralised dashboard makes it easy for traffic operators and engineers to compare models. Random Forest clearly won as the production choice ($R^2 = 0.9334$, lowest MAE of 296.62), while still allowing the team to compare trade-offs against the LSTM's sequence modelling.

- Audit trail and accountability: Public transit and city infrastructure require transparency. MLFlow creates a permanent record of the exact code, features, training settings, and test scores behind every model before it goes live.
- Deployment: Because model files are packaged and their exact input formats, downstream API services or batch scoring jobs can load the production model immediately without having to rebuild parameter pipelines by hand.

4. Technical Limitations & Mitigation

- Single Machine Local Storage: The setup currently runs on a local SQlite database and saves model files to local storage. In a larger production environment with multiple users this becomes an issue as it is a single point of failure, and also will create issues with simultaneous access. This can be mitigated by created a shared cloud database and store model artifacts in the central cloud storage.
- MLflow is designed for tracking training metrics and static files. It does not automatically watch live production data for sensor degradation, seasonal weather changes, or traffic pattern shifts, this can be mitigated by combining MLFlow with dedicated monitoring tools like Prometheus and Grafana to continuously compare incoming feature distributions against the baseline created during the training.

Task 5: Recommendation System Engine — Dynamic Departure & Congestion Mitigation

1. Technique Selection Rationale

- Standalone machine learning models output raw numbers, such as expected vehicle counts or risk probabilities. Without a decision-making layer, these figures do not give commuters or fleet dispatchers clear steps to avoid gridlock
- The engine merges supervised machine learning (Random Forest regression for traffic volume and classification for high accident risk) with unsupervised rule mining (Apriori association patterns). This hybrid approach delivers both forward-looking traffic estimates and historical risk warnings.
- Instead of issuing simple warning flags, the engine evaluates nearby departure options within a ± 2-hour range. It identifies travel times that reduce both traffic congestion and accident exposure.

2. Implementation Methodology

Modular Pipeline Architecture (recommendation_system/recommender.py):

- Reads saved model binaries (best_regression_rf.pkl, best_classification_rf.pkl) and mined association rules (traffic_association_rules.csv)
- Encodes cyclical time inputs ($\sin/\cos$ for hour and day), road visibility flags, and one-hot weather classes so input formats match the original training features. (_build_feature_vector)
- Estimates vehicle volume and calculates high-risk probability ($P(\text{risk}) \geq 0.50$) for the requested travel hour. (assess_conditions)
- Tests candidate departure hours across $t - 2\text{h}$, $t - 1\text{h}$, t, $t + 1\text{h}$, and $t + 2\text{h}$. The system prioritizes lower risk first, followed by lower traffic volume, requiring at least a 15% reduction in traffic to recommend shifting travel plans. (recommend_departure)
- Looks up historical traffic patterns matching the time of day (such as morning or evening rush) and day type (weekday or weekend), displaying empirical confidence and lift scores. (_get_matching_rules)

- **Evaluated Benchmark Scenarios**:

- *Weekday Morning Commute in Rainy Conditions (Mon 08:00, 12°C, Rain)*: Detected peak-hour congestion and tested earlier and later departures to avoid morning bottlenecks
- *Weekday Evening Commute in Heavy Winter Snow (Thu 17:00, -5°C, Snow)*: Identified combined risks from evening rush hour and slick road conditions, recommending schedule shifts to bypass high-risk windows.
- *Weekend Afternoon Leisure Trip (Sun 14:00, 22°C, Clear)*: Confirmed light, off-peak driving conditions, validating that the requested travel window was already optimal and needed no change

3. Business & Operational Value

- By showing exact savings (like leaving two hours earlier to cut traffic by 38.4%), the system convinces drivers to travel during quieter hours.
- By flagging high-risk driving hours, the system helps commuters to avoid dangerous mixes of bad weather and heavy congestion.
- Clear historical patterns and proven rule statistics back every suggestion, making recommendations transparent and easy to trust.

4. Technical Limitations & Mitigation

- The system currently assumes weather stays the same across the entire ± 2-hour evaluation window, which may not hold during fast-moving storms. This can be mitigated by connecting the feature pipeline to a live weather forecast API, so that each candidate departure hour uses its own projected weather.
- If too many people follow the same advice at once, traffic simply shifts and creates a new rush hour. This can be prevented by considering individual driver schedules to spread departures across different times.